Lab 2: Vector Addition

1. Objective

The purpose of this lab is to get you familiar with using the CUDA API by implementing a simple vector addition kernel and its associated setup code.

2. Preliminary work

Step 1: Log in to the GPU side of Roihu (see Lab 1 if the login is refused because your SSH certificate has expired):

```
ssh <username>@roihu-gpu.csc.fi
```

Step 2: If you have not already cloned the lab repository in Lab 1, do it now, in the directory where you would like to put it:

```
git clone https://github.com/jolilius/GP26_Labs.git
cd GP26_Labs
```

You are now ready to begin the lab.

3. Procedure

Step 1: Update your local repository to obtain the code needed for the assignment.

```
cd GP26_Labs
git pull
cd Lab_2
```

Step 2: Edit the file main.cu to implement the following where indicated:

1. Allocate device memory
2. Copy host memory to device
3. Initialize thread block and kernel grid dimensions and invoke CUDA kernel
4. Copy results from device to host
5. Free device memory

Step 3: Edit the file kernel.cu to implement the vector addition kernel code.

Step 4: Compile and test your code. Programs that use a GPU run on a compute node in a Slurm job, so you submit `run.sh`, which builds the program on the node and runs it. Use the CSC project you joined in Lab 1 as the account:

```
sbatch --account=<your-project> run.sh       # Uses the default vector size
sbatch --account=<your-project> run.sh <m>   # Uses vectors of size m
```

The output is written to `lab2_<jobnumber>.out` and to `output.txt` (each job overwrites `output.txt`). Wait for the job to finish (`squeue --me` shows your jobs) and then read the file. A correct solution prints something like the following (the timings will differ):

```
Setting up the problem...0.000293 s
    Vector size = 10000
Allocating device variables...0.637861 s
Copying data from host to device...0.000045 s
Launching kernel...0.000106 s
Copying data from device to host...0.000026 s
Verifying results...TEST PASSED
```

Compile errors show up in `lab2_<jobnumber>.out`.

It is a good idea to test with several sizes, including sizes that are not a multiple of your block size, and then to run the default size last so that `output.txt` holds the output for the default size.

Step 5: Submit your assignment. You only need to submit the following files:

- main.cu
- kernel.cu
- output.txt (the output of your run, which must show `TEST PASSED`)

Compress the files and name them after your student name and student id like so:

```
Name_id.tar
```

Upload the file to the assignment in Moodle.